



21CSC304T
ARTIFICIAL
INTELLIGENCE

Unit III

1. **Game Theory:** Adversarial Search Methods - Minimax algorithm - Alpha beta pruning.
1. **Constraint satisfactory problems:** Constraints – Crypt Arithmetic Puzzles – Constraint Domain – CSP as a search problem (Room coloring).
1. **Agents:** Intelligent Agent – Rationality and Rational Agent – Performance Measures – Rationality and Performance – Flexibility and Intelligent Agents – Task environment and its properties – Types of agents.

GAME THEORY



- Adversarial Search Methods
 - a. Mini max algorithm
 - b. Alpha beta pruning.

Adversarial search Methods

- **Adversarial search** is a search, where we examine the problem which arises when we try to plan ahead of the world and other agents are planning against us.
- ***According to game theory, a game is played between two players. To complete the game, one has to win the game and the other loses automatically.***
- Adversarial search: Search based on **Game theory** - Agents- Competitive environment
- **Conflicting goal** - adversarial search



We are opponents- I win, you loose.

General Search Vs Adversarial Search

- In previous topics, we have studied the **search strategies** which are only associated with a **single agent** that aims to **find the solution** which often **expressed in the form of a sequence of actions**.
- But, there might be some **situations where more than one agent is searching for the solution** in the same search space, and this situation usually **occurs in game playing**.
- The environment with **more than one agent** is termed as **multi-agent environment**, in which each agent is an opponent of other agent and playing against each other. Each agent needs to consider the action of other agent and effect of that action on their performance.
- So, **Searches in which two or more players with conflicting goals are trying to explore the same search space for the solution, are called adversarial searches, often known as Games**.
- Games are modeled as a **Search problem and heuristic evaluation function**, and these are the two main factors which help to model and solve games in AI.

Elements of Game playing

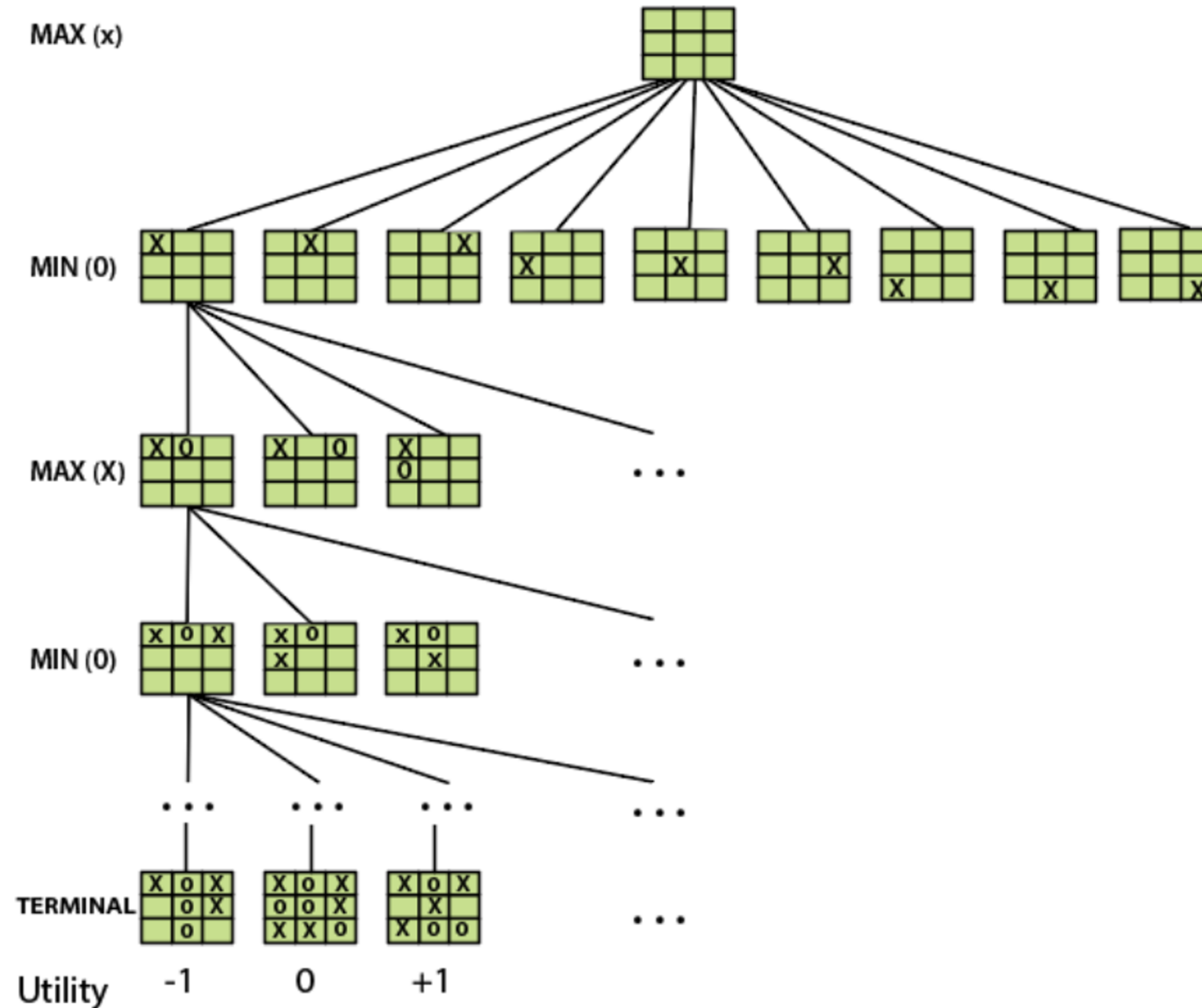
```
 $s_0$  //initial state  
player(s) // who's the player in state  $s$   
actions(s) // possible moves from state  $s$   
result(s,a) // the state after action  $a$  is taken on state  $s$   
terminal(s) // returns true if  $s$  is a terminal state  
utility(s,p) // the objective function in state  $s$  for player  $p$ 
```

(-1): If the PLAYER loses.

(+1): If the PLAYER wins.

(0): If there is a draw between the PLAYERS.

Game Tree for Tic tac toe



-
- MAX (x)
- MIN (O)
- MAX (X)
- MIN (O)
- TERMINAL
- Utility
- 1 0 +1

Techniques - To get best optimal solution

There is always a **need to choose algorithms that provide best optimal solution in limited time**. Techniques that fulfill this requirement are as following:

1. **Pruning:** Allows **ignoring the unwanted portions of a search tree** which makes no difference in the final solution.
2. **Heuristic Evaluation Function:** Allows to **approximate the cost value at each level of the search tree** before reaching a goal node.

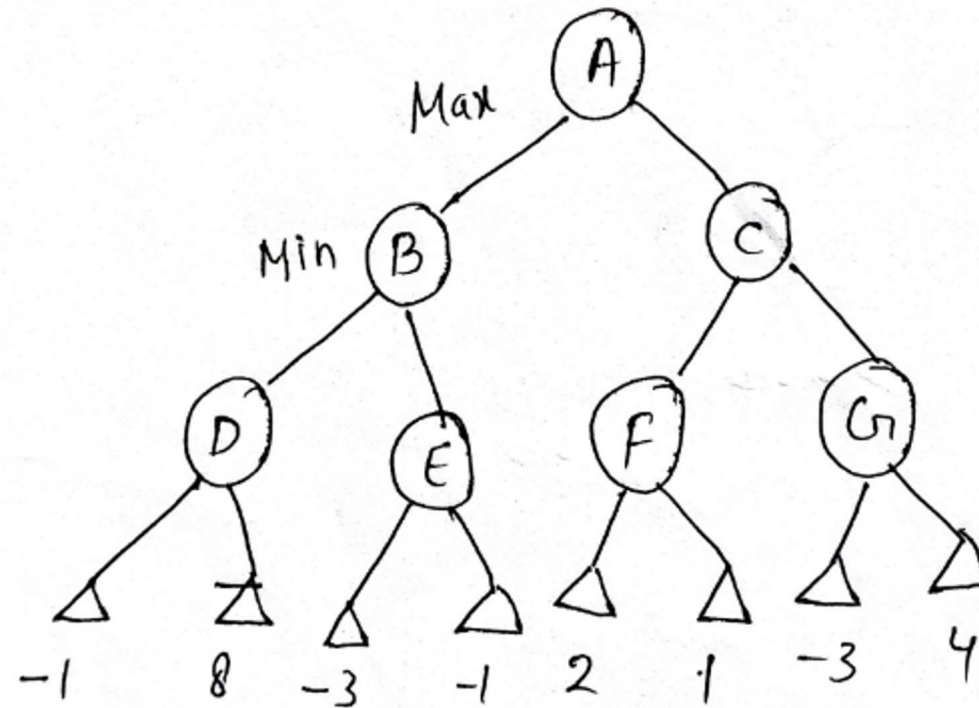
Game as a search problem

- **Types of algorithms in Adversarial search**
 - In a **normal search**, we follow a sequence of actions to reach the goal or to finish the game optimally. But in an **adversarial search**, the result **depends on the players** which will decide the result of the game. It is also obvious that the solution for the goal state will be an optimal solution because the player will try to win the game with the shortest path and under limited time.
 - **Minmax Algorithm**
 - **Alpha-beta Pruning**

Minimax Algorithm

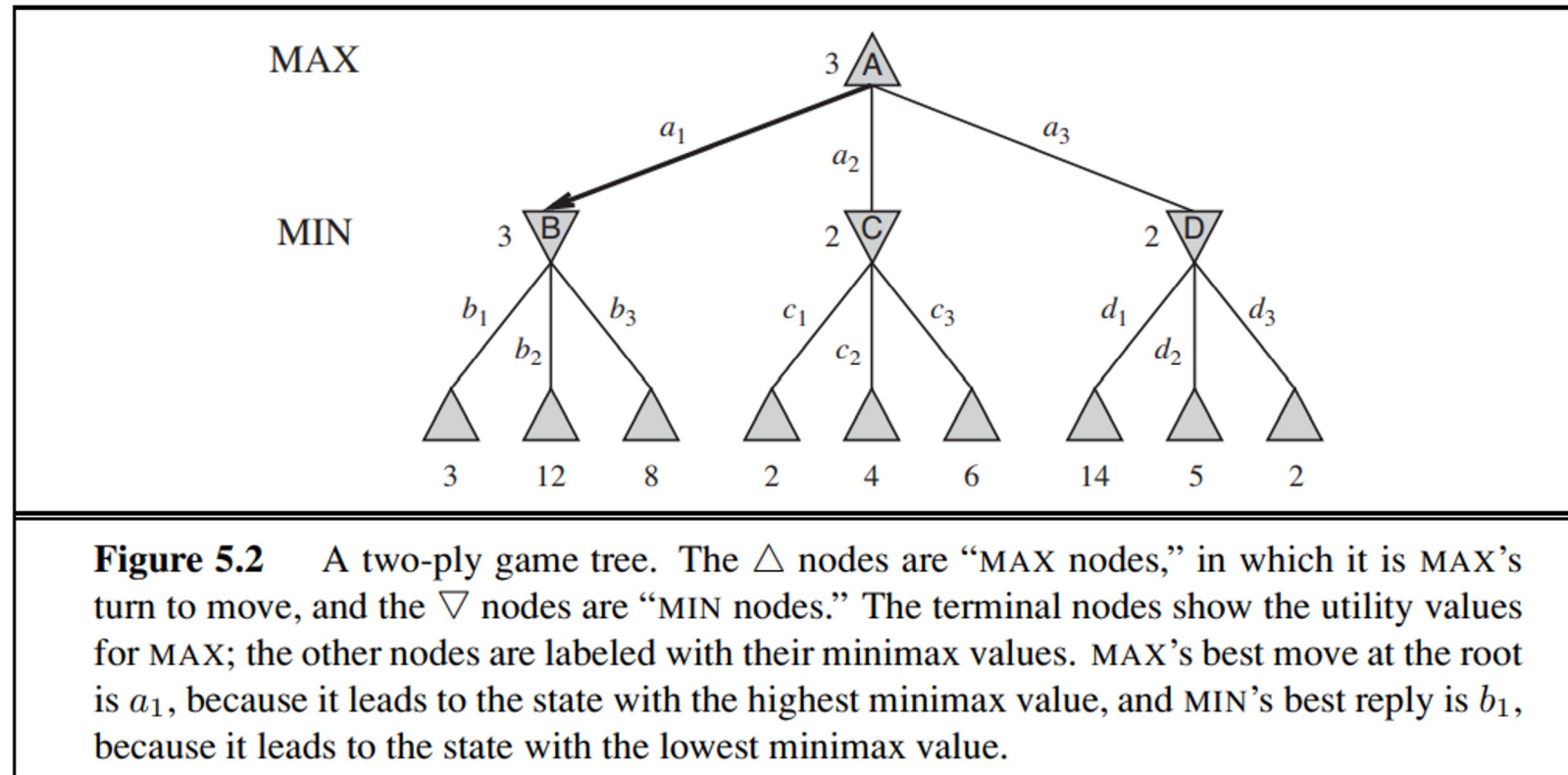
- **Minimax/Minmax/MM/Saddle Point:**
 - **Decision strategy** - Game Theory
 - Minimize losing chances in a game and Maximize winning chances
 - **Two-player game** strategy. *Ex. Chess*
 - Represented as **game tree**- where the ***nodes** represent the **states of the game** and **edges** represent the **moves made by the players***
 - Players will be two namely:
 - **MIN:** Decrease the chances of **MAX** to win the game.
 - **MAX:** Increases his chances of winning the game.
 - Result of the game/**Utility value**
 - Generated by **Heuristic function** propagating from initial node to root node
 - **Backtracking technique** - to find the Best choice
 - MAX will choose that path which will increase its utility value and MIN will choose the opposite path which could help it to minimize MAX's utility value.

Minmax Algorithm - Example



Game tree

Minimax Approach



MINIMAX(s) =

$$\begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

Properties of Mini-Max algorithm:

- **Complete-** Min-Max algorithm is Complete. It will definitely find a solution (if exist), in the finite search tree.
- **Optimal-** Min-Max algorithm is optimal if both opponents are playing optimally.
- **Time complexity-** As it performs DFS for the game-tree, so the time complexity of Min-Max algorithm is $O(b^m)$, where b is branching factor of the game-tree, and m is the maximum depth of the tree.
- **Space Complexity-** Space complexity of Mini-max algorithm is also similar to DFS which is $O(bm)$.

Limitation of the minimax Algorithm:

- The main drawback of the minimax algorithm is that it **gets really slow for complex games such as Chess, go, etc.**
- This type of games has a **huge branching factor**, and the **player has lots of choices to decide.**
- This limitation of the minimax algorithm can be improved from **alpha-beta pruning**

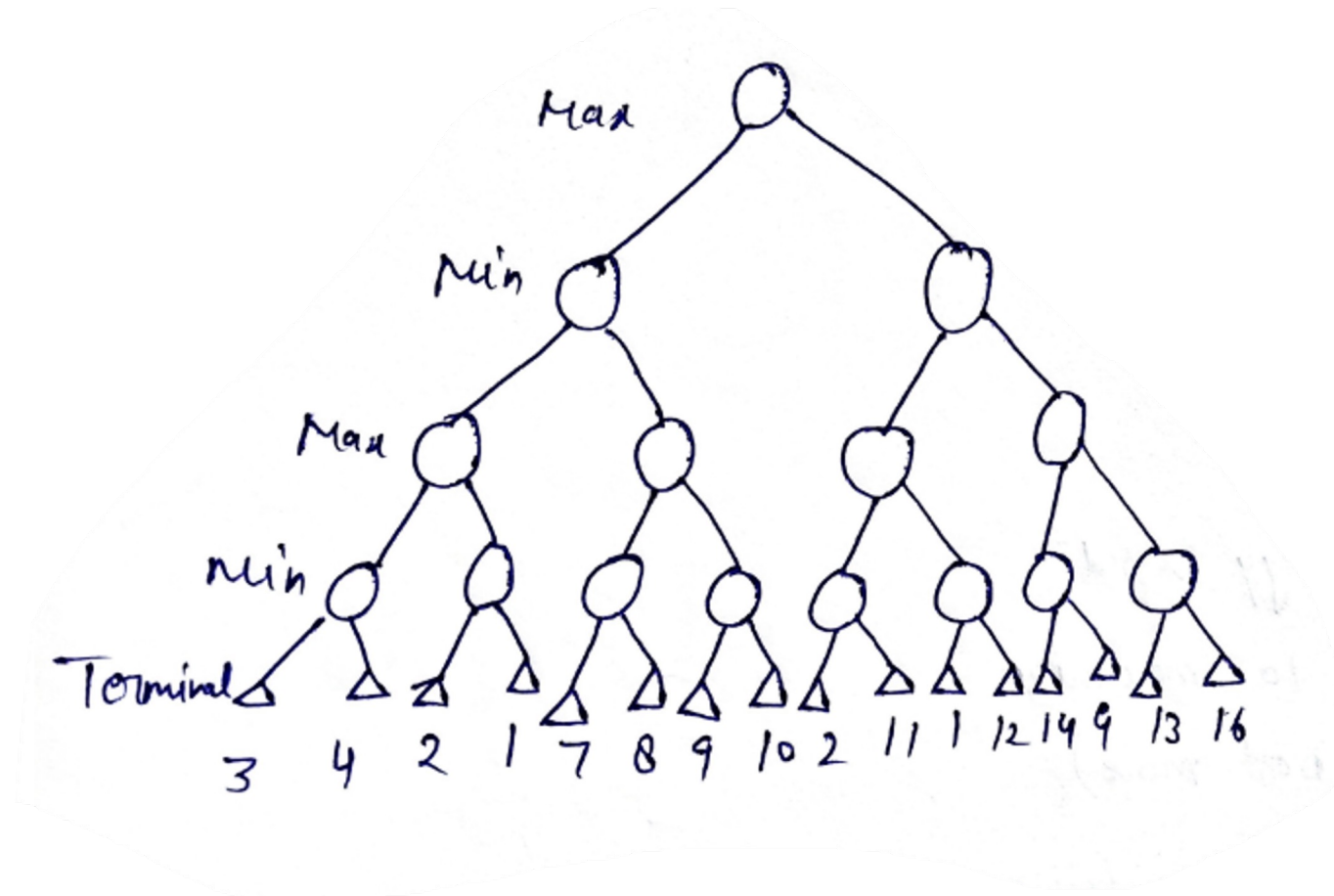
Alpha beta pruning

- Advanced version of MINIMAX algorithm
- Any optimization algorithm- performance measure is the first consideration
- **Drawback of Minimax:**
 - Explores each node in the tree deeply to provide the best path among all the paths
 - Increases time complexity
- **Alpha beta pruning:** Overcomes drawback by less exploring nodes of search tree

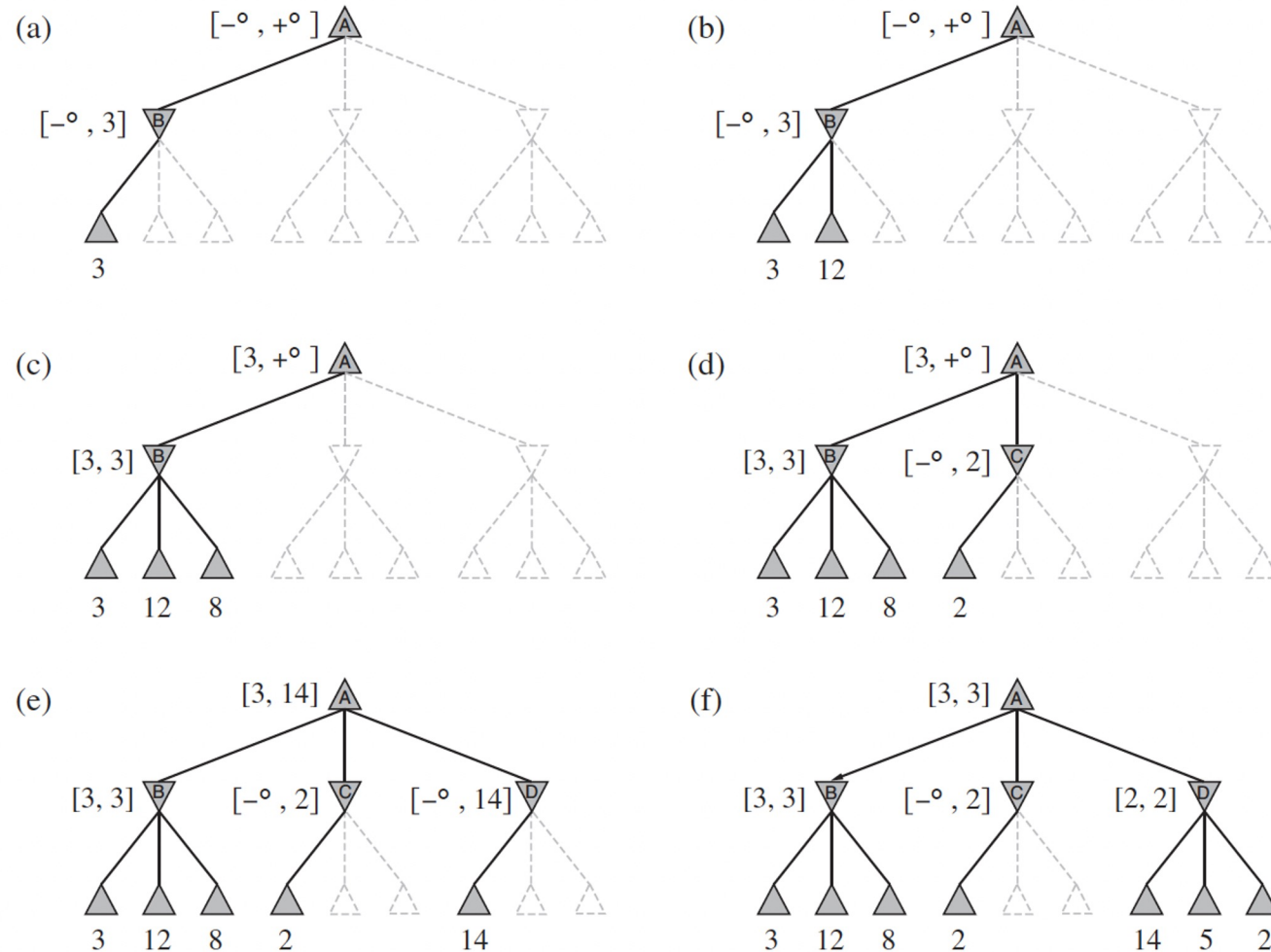
Alpha beta pruning

- Cut off the search by exploring less number of nodes
- It makes same moves as Minimax algorithm does- but prunes unwanted branches using the pruning techniques
- Alpha beta pruning works on 2 threshold values α and β
 - α : It is the best highest value, a **MAX** player can have. It is the lower bound, which represents negative infinity value.
 - β : It is the best lowest value, a **MIN** player can have. It is the upper bound which represents positive infinity.
- So, each MAX node has α -value, which never decreases, and each MIN node has β -value, which never increases.
- **Note:** Alpha-beta pruning technique can be applied to trees of any depth, and it is possible to prune the entire subtrees easily.

Alpha beta pruning: Example

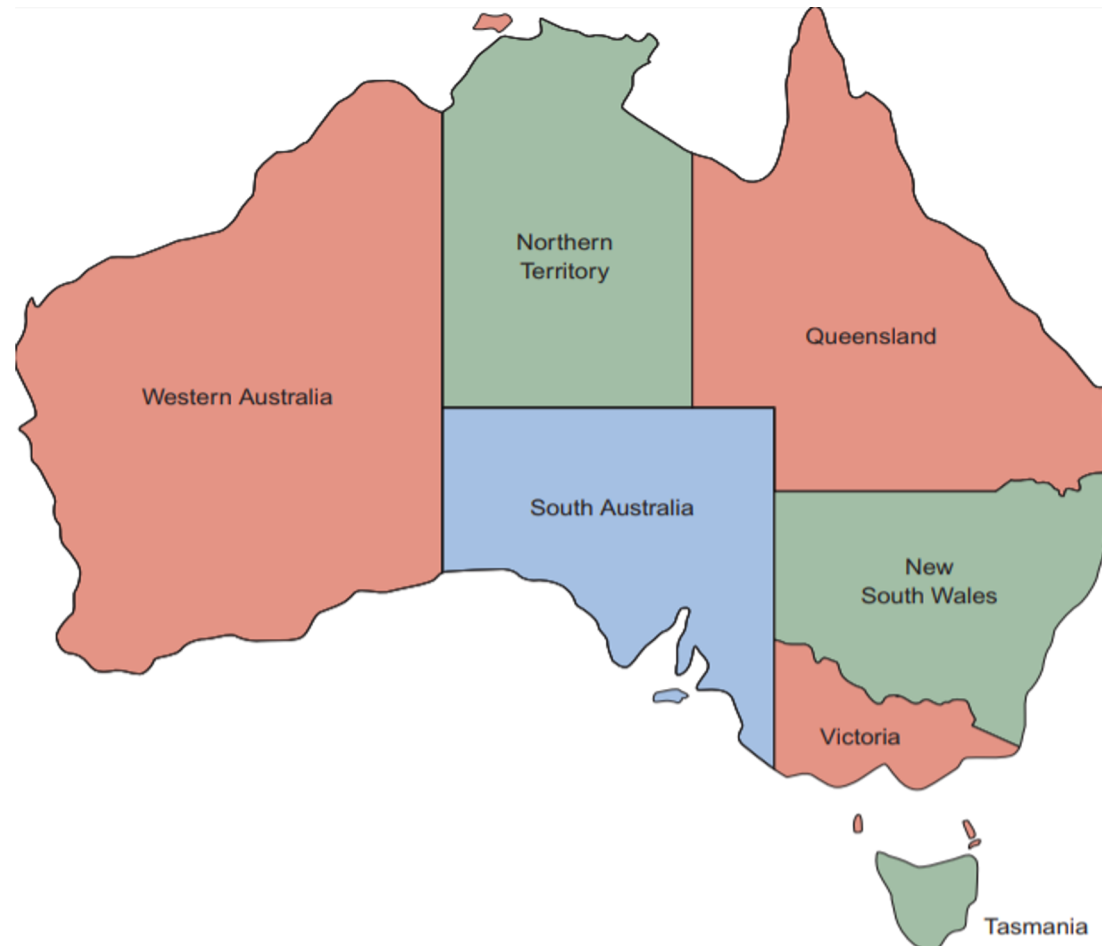


Example Alpha beta pruning



Stages in the calculation of the optimal decision for the game tree

Constraint Satisfaction Problem



- Constraint Satisfaction problems
- Constraints
- Cryptarithmic Puzzles
- Constraint Domain
- CSP as a search problem (Rooming coloring).

Constraint Satisfaction Problems (CSP)

- The objective of every problem-solving technique is one, i.e., **to find a solution to reach the goal.**
- Search Space is constrained by a set of conditions and dependencies
- Solving a problem under **certain constraints or rules.**
- Class of problems where **the search space is constrained** called as **CSP**
- For example, Time-table scheduling problem for lecturers.
- Constraint,
 - Two Lecturers can not be assigned to same class at the same time
- To solve CSP, the problem is to be decomposed and analyze the structure
- Constraints are typically mathematical or logical relationships

Constraint Satisfaction Problem

A constraint satisfaction problem consists of **three components, X, D, and C**:

- X is a set of variables, $\{X_1, \dots, X_n\}$.
 - D is a set of domains, $\{D_1, \dots, D_n\}$, one for each variable.
 - C is a set of constraints that specify allowable combinations of values.
-
- Each **domain D_i** consists of a **set of allowable values**, $\{v_1, \dots, v_k\}$ for variable X_i .
 - Each **constraint C_i** consists of a pair **scope, rel**, where scope is a tuple of variables that participate in the constraint and rel is a relation that defines the values that those variables can take on.
- Ex: Sudoku (9X9 board):
1. Set of variables: $9 \times 9 = 81$
 2. Domain: 1-9
 3. Constraints: No number should repeat twice in the same row or column.

Constraint Types in CSP

- With respect to the variables, basically there are following types of constraints:
 - **Unary Constraints:** It is the simplest type of constraints that restricts the value of a **single variable**.
 - **Binary Constraints:** It is the constraint type which relates **two variables**. A value x_2 will contain a value which lies between x_1 and x_3 .
 - **Global Constraints:** It is the constraint type which involves an **arbitrary number of variables**.

Real World Examples

One interesting CSP in everyday life is loading of boxes in a truck. Here, all boxes of different shapes are to be loaded in a container. The constraints can be as follows:

1. Boxes of more weight cannot be put on that of lower weight.
2. Boxes with fragile material should always be kept on top.
3. Boxes with shape other than cubical and cuboids should not be kept at bottom.
4. A box with the largest surface area should be kept at bottom.

CSP : Another Example

- ▶ Any problem in the world can be mathematically represented as CSP
- ▶ Hypothetical problem does not have constraints
- ▶ Constraint restricts movement, arrangement, possibilities and solutions
- ▶ Example: if only concurrency notes of 2,5,10 rupees are available and we need to give certain amount of money, say Rs.111 to a salesman and the total number of notes should be between 40 and 50.
- ▶ Expressed as, Let n be number of notes,

$$40 < n < 50$$

and

$$c_1X_1 + c_2X_2 + c_3X_3 = 111$$

Real-world CSPs

- Cryptarithmic problem
- Room colouring problem
- Map/Graph colouring problem
- Cross-word puzzle
- Sudoku Game
- Assignment problems
 - e.g., who teaches what class
- Timetabling problems
 - e.g., which class is offered when and where?
- Transportation scheduling
- Factory scheduling

Crypt-arithmetic Puzzles

- ▶ Cryptarithmic puzzles are also represented as CSP
- ▶ Example: SEND + MORE = MONEY
- ▶ Replace every letter in puzzle with single number (number should not be repeated for two different alphabets)
- ▶ The domain is $\{ 0, 1, \dots, 9 \}$
- ▶ Often treated as the ten-variable constraint problem where the constraints are:
 - ▶ All the variables should have a different value
 - ▶ The sum must work out
 - ▶ The carry forward should always be '1'

Cryptarithmic Problem

$$\begin{array}{r}
 \text{S} \quad \text{E} \quad \text{N} \quad \text{D} \\
 + \quad \text{M} \quad \text{O} \quad \text{R} \quad \text{E} \\
 \hline
 \text{M} \quad \text{O} \quad \text{N} \quad \text{E} \quad \text{Y}
 \end{array}$$

$$\begin{array}{r}
 \square \quad \square \quad \square \quad \square \\
 + \quad \square \quad \square \quad \square \quad \square \\
 \hline
 \square \quad \square \quad \square \quad \square \quad \square
 \end{array}$$

Character	code
S	
E	
N	
D	
M	
O	
R	
Y	

Room Coloring Problem

- ▶ Let *K* for Kitchen, *D* for Dining Room, *H* is for Hall, *B*₂ and *B*₃ are bedrooms 2 and 3, *MB*₁ is master bedroom, *SR* is the store Room, *GR* is Guest Room and *Lib* is Library
- ▶ Constraints
 - ▶ All bedrooms should not be colored red, only one can
 - ▶ No two adjacent rooms can have the same color
 - ▶ The colors available are red, blue, green and violet
 - ▶ Kitchen should not be colored green
 - ▶ Recommended to color the kitchen as blue
 - ▶ Dining room should not have violet color

Room Coloring Problem

- ▶ Let K for Kitchen, D for Dining Room, H is for Hall, B_2 and B_3 are bedrooms 2 and 3, MB_1 is master bedroom, SR is the store Room, GR is Guest Room and Lib is Library
- ▶ Constraints
 - ▶ All bedrooms should not be colored red, only one can
 - ▶ No two adjacent rooms can have the same color
 - ▶ The colors available are red, blue, green and violet
 - ▶ Kitchen should not be colored green
 - ▶ Recommended to color the kitchen as blue
 - ▶ Dining room should not have violet color

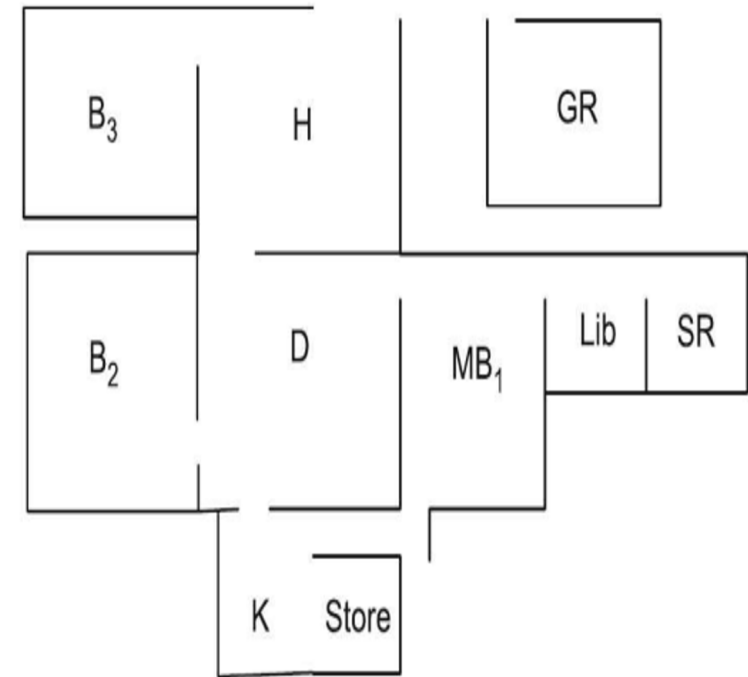


Figure 6.3 Home colouring problem.

Representation as a search problem

Constraint Graph Construction

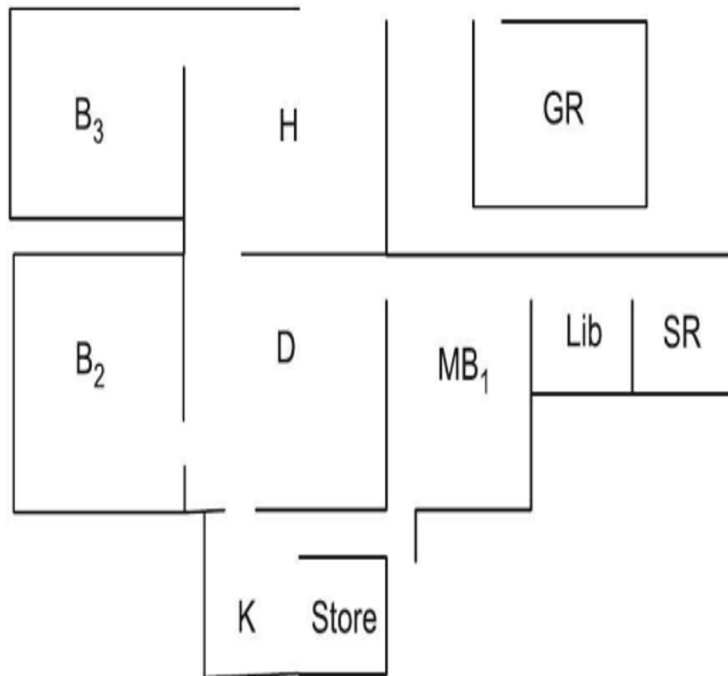


Figure 6.3 Home colouring problem.

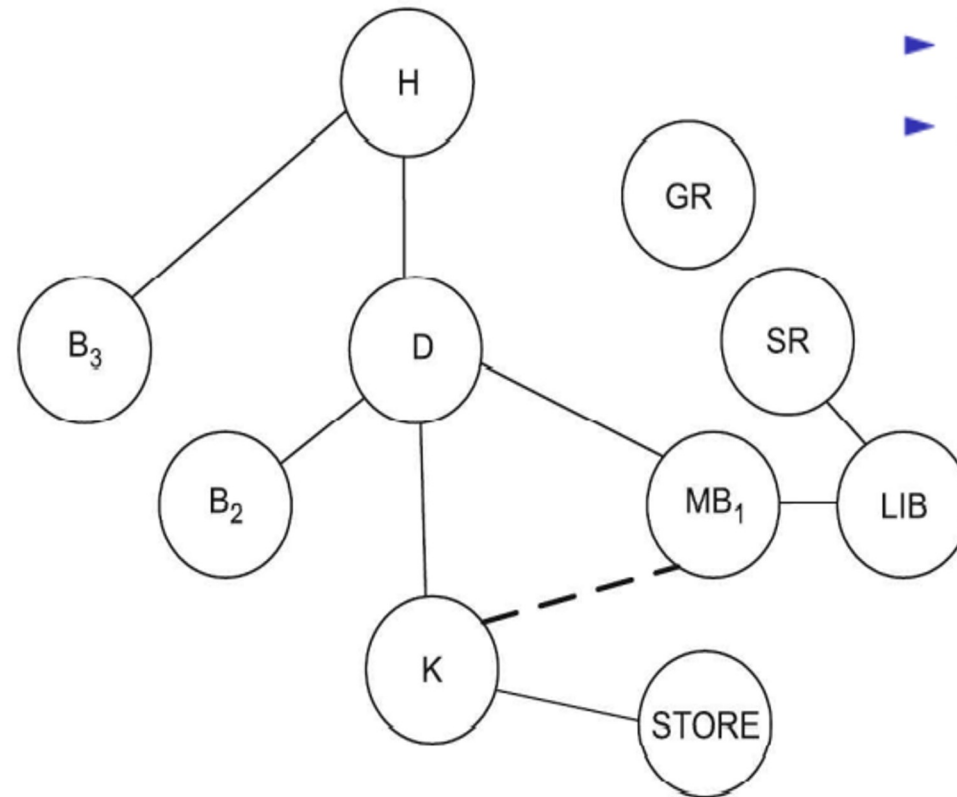
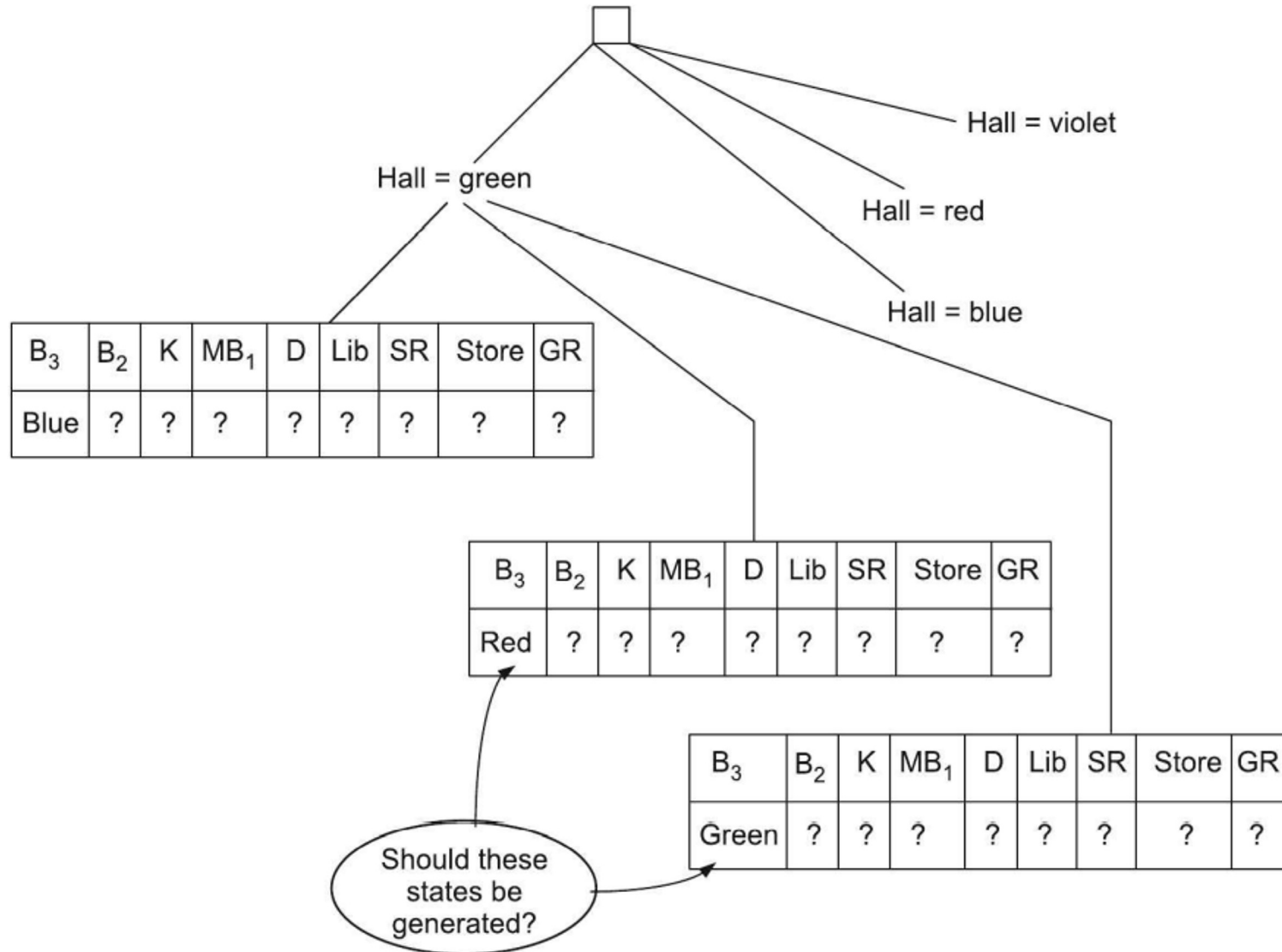
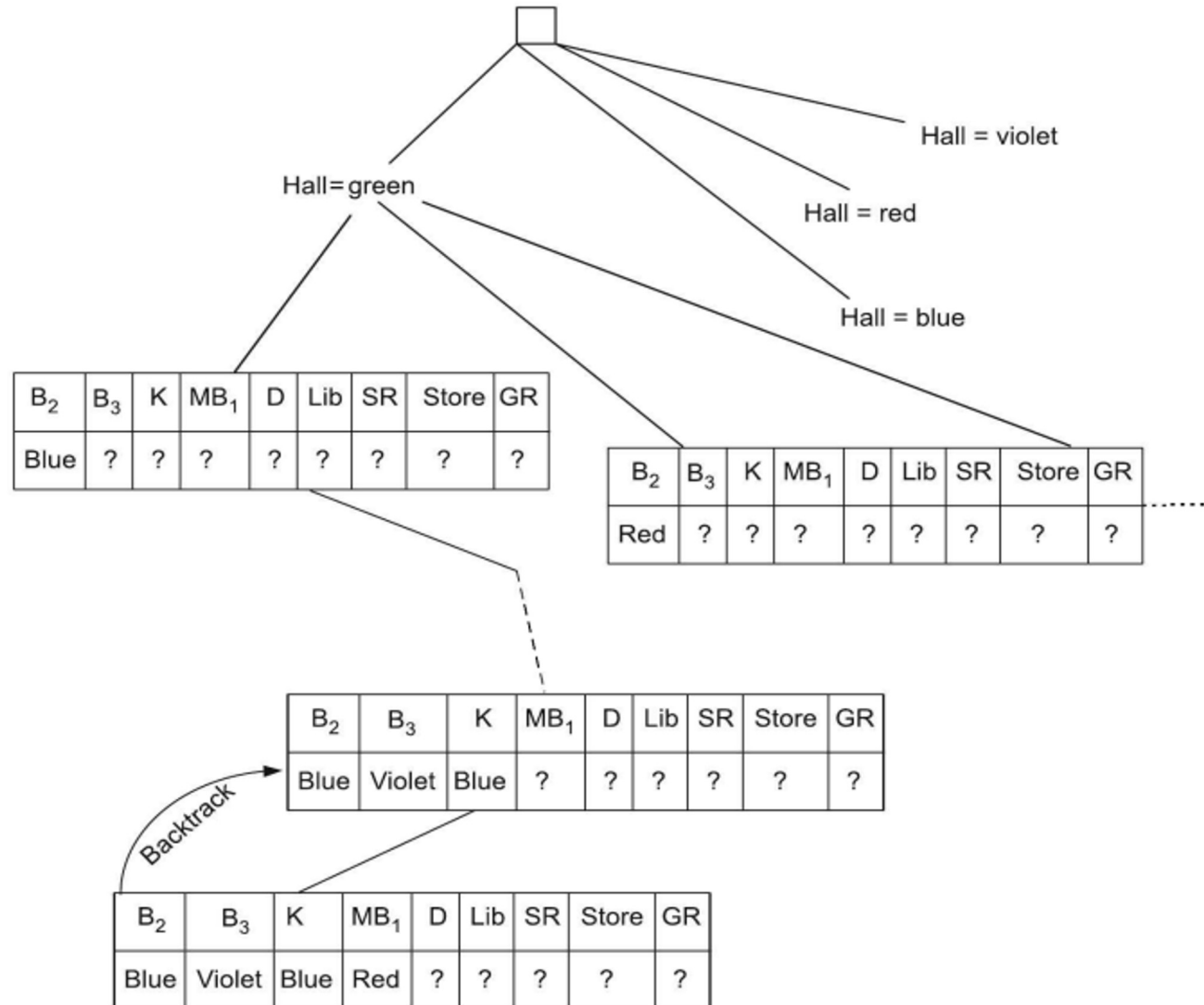


Figure 6.4 Constraint graph.

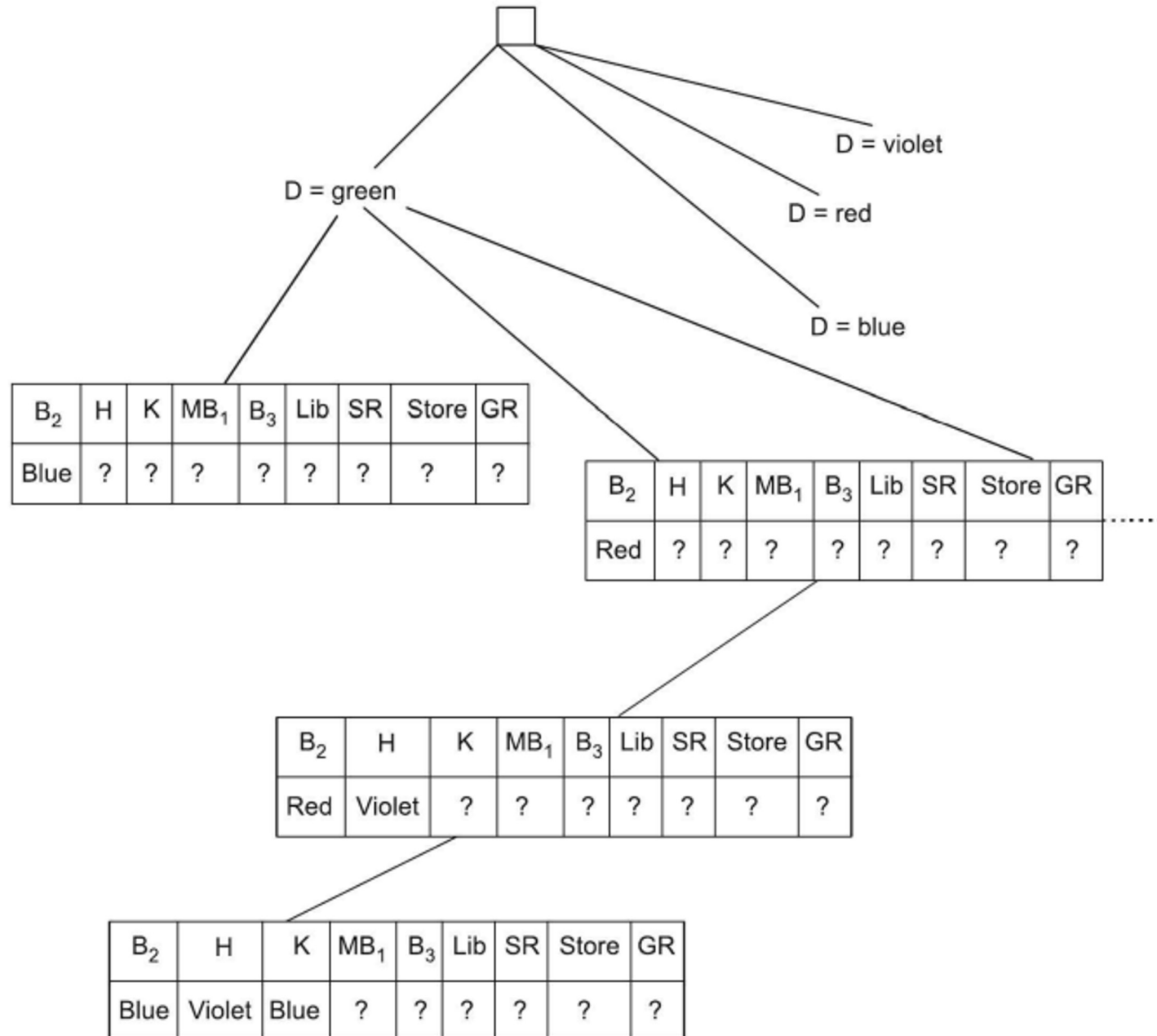
Search tree Generation



Backtracking Search for CSP



- At a stage when a color needs to be assigned to dining area, no options are available due to constraint violation
- There is a need to backtrack
- In DFS level wise backtracking occurs
- In CSP, backtracks directly where conflict occurs, hence called **intelligent backtracking**



AGENTS



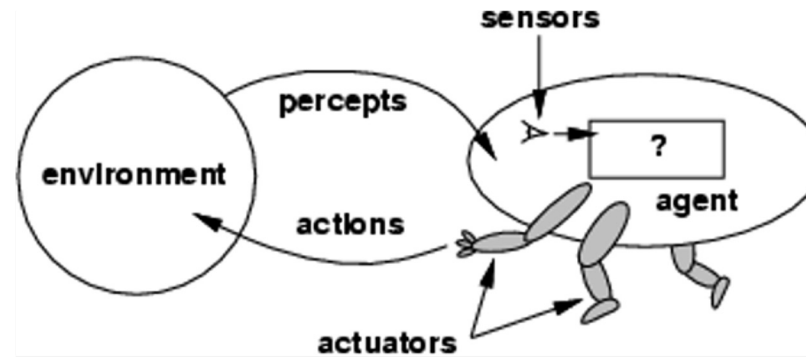
Agents

- An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators
- **Human agent:**
 - eyes, ears, and other organs for sensors;
 - hands, legs, mouth, and other body parts for actuators
- **Robotic agent:**
 - cameras and infrared range finders for sensors
 - various motors for actuators



- Intelligent Agent
- Rationality and Rational Agent
- Performance Measures
- Rationality and Performance
- Flexibility and Intelligent Agents
- Task environment and its properties
- Types of agents.

Agents and environments

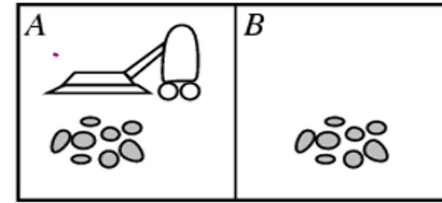


- The agent function maps from percept histories to actions:

$$[f: P^* \rightarrow A]$$

- The agent program runs on the physical architecture to produce f
- agent = architecture + program

Vacuum-cleaner world



- Percepts: location and contents, e.g., [A,Dirty]
- Actions: *Left, Right, Suck, NoOp*
- *Agent's function* → *look-up table*
 - *For many agents this is a very large table*

Percept sequence	Action
[A, Clean]	<i>Right</i>
[A, Dirty]	<i>Suck</i>
[B, Clean]	<i>Left</i>
[B, Dirty]	<i>Suck</i>
[A, Clean], [A, Clean]	<i>Right</i>
[A, Clean], [A, Dirty]	<i>Suck</i>
⋮	⋮

Rational Agent

- A rational agent is an agent which has **clear preference, models uncertainty, and acts in a way to maximize its performance measure** with **all possible actions**.
- A rational agent is said to **perform the right things**. AI is about creating rational agents to use for game theory and decision theory for various real-world scenarios.
- For an AI agent, the rational action is most important because in AI reinforcement learning algorithm, for each best possible action, agent gets the positive reward and for each wrong action, an agent gets a negative reward.

Rationality

- An ideal **rational agent** should, for each possible percept sequence, do whatever actions will maximize its expected performance measure based on
 - (1) the percept sequence, and
 - (2) its built-in and acquired knowledge.
- Rationality includes information gathering, not “rational ignorance.” (If you don't know something, find out!)
- Rationality → Need a performance measure to say how well a task has been achieved.
- Types of performance measures: false alarm (false positive) and false dismissal (false negative) rates, speed, resources required, effect on environment, etc.

Rationality and Performance

- ▶ Rationality is with knowledge available with the agent
- ▶ Knowledge is derived from the percept sequence to date
- ▶ Learning agent which is part of rational agent that can succeed in variety of environments
- ▶ Rational agent should possess the following properties
 - ▶ Ability to gather information
 - ▶ Ability to learn from the experience
 - ▶ Perform knowledge augmentation
 - ▶ Autonomy

Performance measure

- The criteria that determine how successful an agent is.
- One fixed measure is not suitable for all agents
- Performance measures: Safe, fast, legal, comfortable trip, maximize profits
- Example:
 - Agent: Interactive English tutor
 - Performance measure: Maximize student's score on test
 - Environment: Set of students
 - Actuators: Screen display (exercises, suggestions, corrections)
 - Sensors: Keyboard

Flexibility

- Equips agent to negotiate with dynamic scenario, i.e., system should be able to adapt with the changing scenario and should exhibit rational behaviour in the changing scenario.
- A system without autonomy lacks flexibility
- Autonomy means “Independence”
 1. **Responsive:** It should respond in timely fashion to the perceived environment.
 2. **Pro-active:** It should exhibit opportunistic, goal-directed behaviour and take the initiative, wherever necessary.
 3. **Social:** It should be able to interact when they are deemed appropriate with the other artificial agents, or humans in order to compete problem solving.

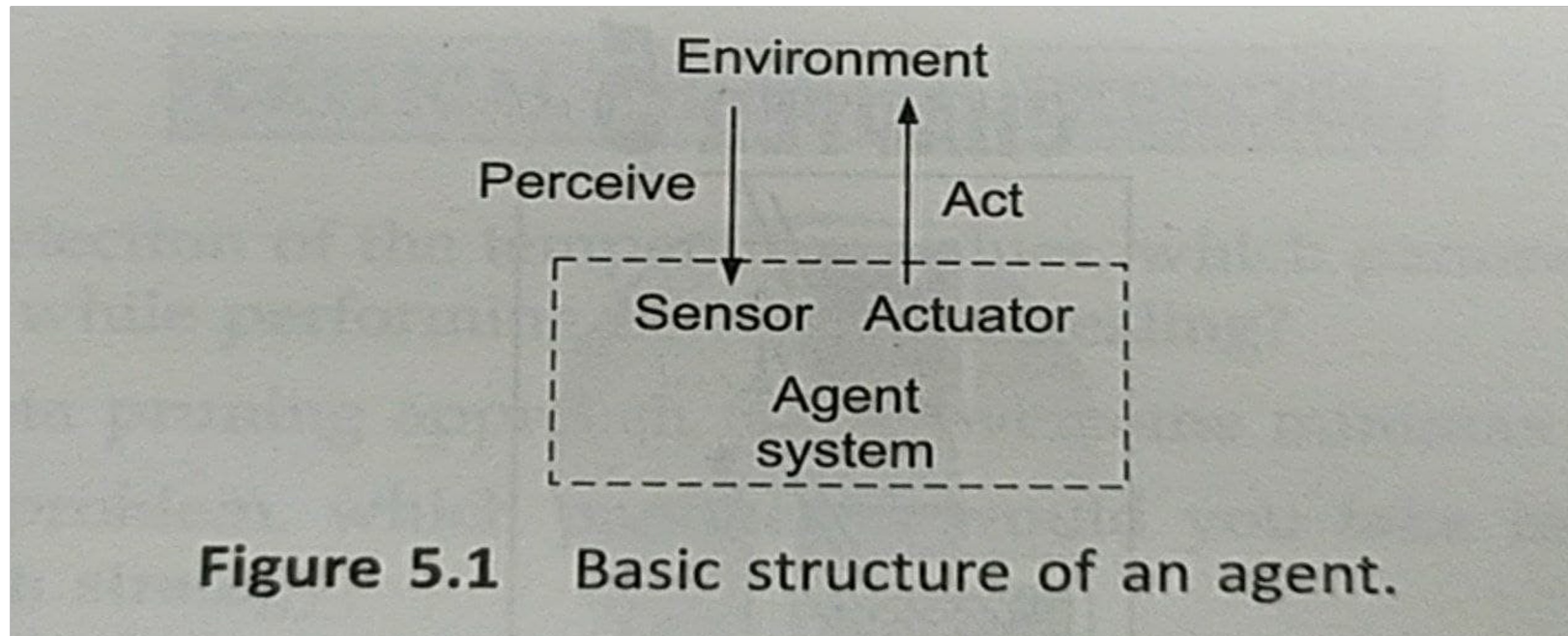
Autonomy in Agents

- The **autonomy** of an agent is the extent to which its behaviour is determined by its own experience, rather than knowledge of designer.
- Extremes
 - No autonomy – ignores environment/data
 - Complete autonomy – must act randomly/no program
- Example: baby learning to crawl
- Ideal: design agents to have some autonomy
 - Possibly become more autonomous with experience

Intelligent Agents

- An intelligent agent is an **autonomous entity which act upon an environment** using **sensors and actuators for achieving goals**.
- An intelligent agent may **learn from the environment to achieve their goals**.
- Following are the main four rules for an AI agent:
 - **Rule 1:** An AI agent must have the ability to perceive the environment.
 - **Rule 2:** The observation must be used to make decisions.
 - **Rule 3:** Decision should result in an action.
 - **Rule 4:** The action taken by an AI agent must be a rational action.

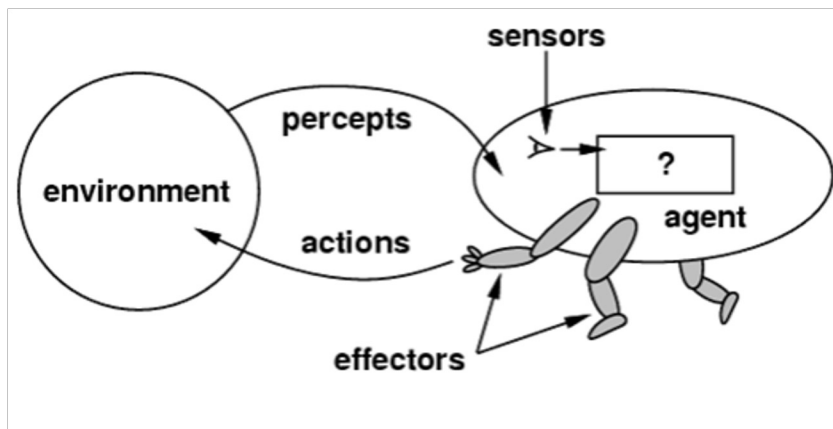
Intelligent Agent



- ▶ Agent - Entity that can perceive the information and act on that information to achieve the desired outcome
- ▶ Intelligent Agent - Capable of making decisions and act most logically
- ▶ Agent Interacts with environment through sensors and actuators

Examples

1. Consider an automatic attendance system. In the system, fingerprint sensor senses the fingerprints, actuator gives the message and the attendance is marked for the employee corresponding to sensed fingerprint.
2. Auto door opening and closing system, a camera is used as sensor for sensing the existence of an object. When the camera senses that a person is standing in front of the door or near the door, it gives this input to the agent program (percepts). Agent gives command and then the actuator opens the door.



- Humans
 - Sensors: Eyes (vision), ears (hearing), skin (touch), tongue (gustation), nose (olfaction), neuromuscular system (proprioception)
 - Percepts:
 - At the lowest level – electrical signals from these sensors
 - After preprocessing – objects in the visual field (location, textures, colors, ...), auditory streams (pitch, loudness, direction), ...
 - Effectors: limbs, digits, eyes, tongue, ...
 - Actions: lift a finger, turn left, walk, run, carry an object, ...
- The Point: percepts and actions need to be carefully defined, possibly at different levels of abstraction

PEAS Representation

PEAS is a type of model on which an AI agent works upon. When we define an AI agent or rational agent, then we can group its properties under PEAS representation model. It is made up of four words:

- **P:** Performance measure
- **E:** Environment
- **A:** Actuators
- **S:** Sensors

Here performance measure is the objective for the success of an agent's behavior.

A more specific example: Automated taxi driving system

- **Percepts:**
- **Actions:**
- **Goals:**
- **Environment:**



PEAS for self-driving cars

Let's suppose a self-driving car then PEAS representation will be:

Performance: Safety, time, legal drive, comfort

Environment: Roads, other vehicles, road signs, pedestrian

Actuators: Steering, accelerator, brake, signal, horn

Sensors: Camera, GPS, speedometer, odometer, accelerometer, sonar.

Agents with PEAS representation

Agent	Performance measure	Environment	Actuators	Sensors
1. Medical Diagnose	• Healthy patient • Minimized cost	• Patient • Hospital • Staff	• Tests • Treatments	Keyboard (Entry of symptoms)
2. Vacuum Cleaner	• Cleanness • Efficiency • Battery life • Security	• Room • Table • Wood floor • Carpet • Various obstacles	• Wheels • Brushes • Vacuum Extractor	• Camera • Dirt detection sensor • Cliff sensor • Bump Sensor • Infrared Wall Sensor
3. Part -picking Robot	• Percentage of parts in correct bins.	• Conveyor belt with parts, • Bins	• Jointed Arms • Hand	• Camera • Joint angle sensors.

Types of Agents

- ▶ Types based on Complexity and functionality (Expected Intelligence) of agent
- ▶ Complexity of the environment contributes to complexity of the agent architecture
- ▶ Types of Agents are,
 - ▶ Table-driven agents
 - ▶ Simple Reflex Agents
 - ▶ Model-based reflex Agents
 - ▶ Goal-based Agents
 - ▶ Utility-based Agents

Some agent types

- **(0) Table-driven agents**
 - use a percept sequence/action table in memory to find the next action. They are implemented by a (large) lookup table.
- **(1) Simple reflex agents**
 - are based on condition-action rules, implemented with an appropriate production system. They are stateless devices which do not have memory of past world states.
- **(2) Agents with memory**
 - have internal state, which is used to keep track of past states of the world.
- **(3) Agents with goals**
 - are agents that, in addition to state information, have **goal information** that describes desirable situations. Agents of this kind take future events into consideration.
- **(4) Utility-based agents**
 - base their decisions on **classic axiomatic utility** theory in order to act rationally.

Table-driven agents (revised from R&N)

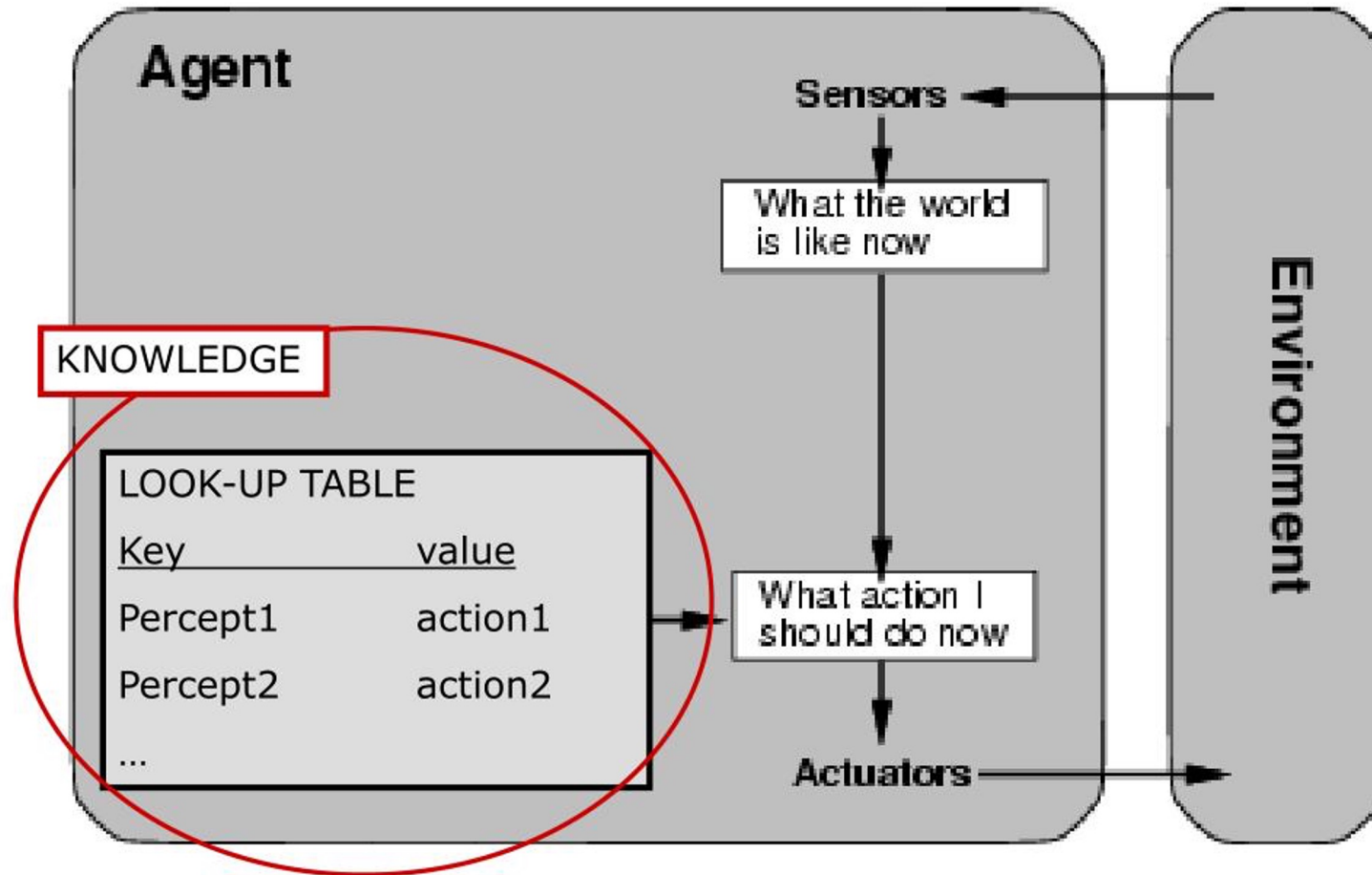
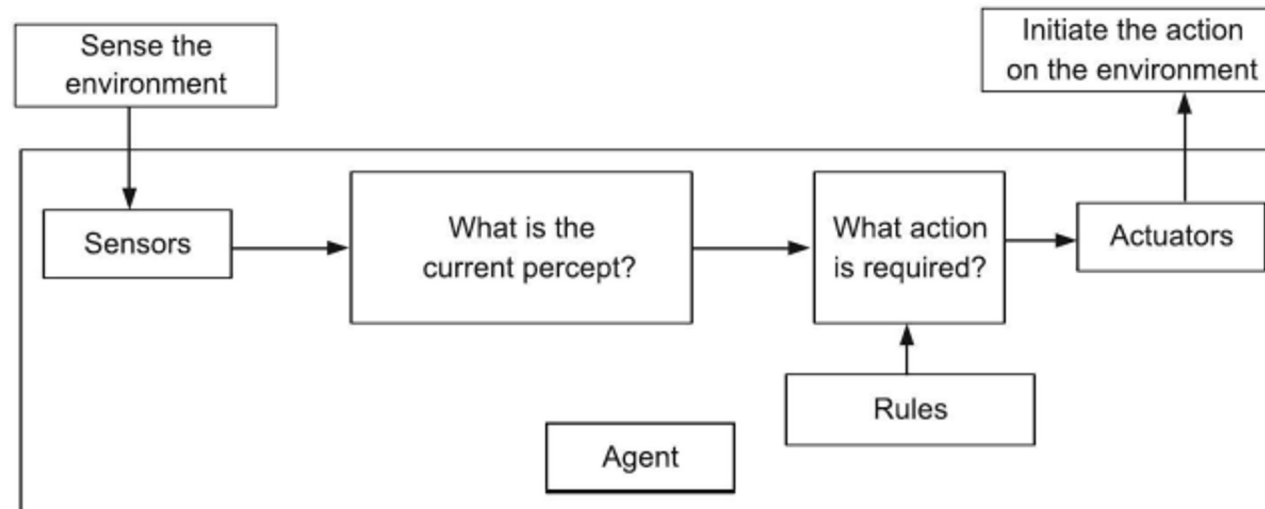


Table-driven Agents

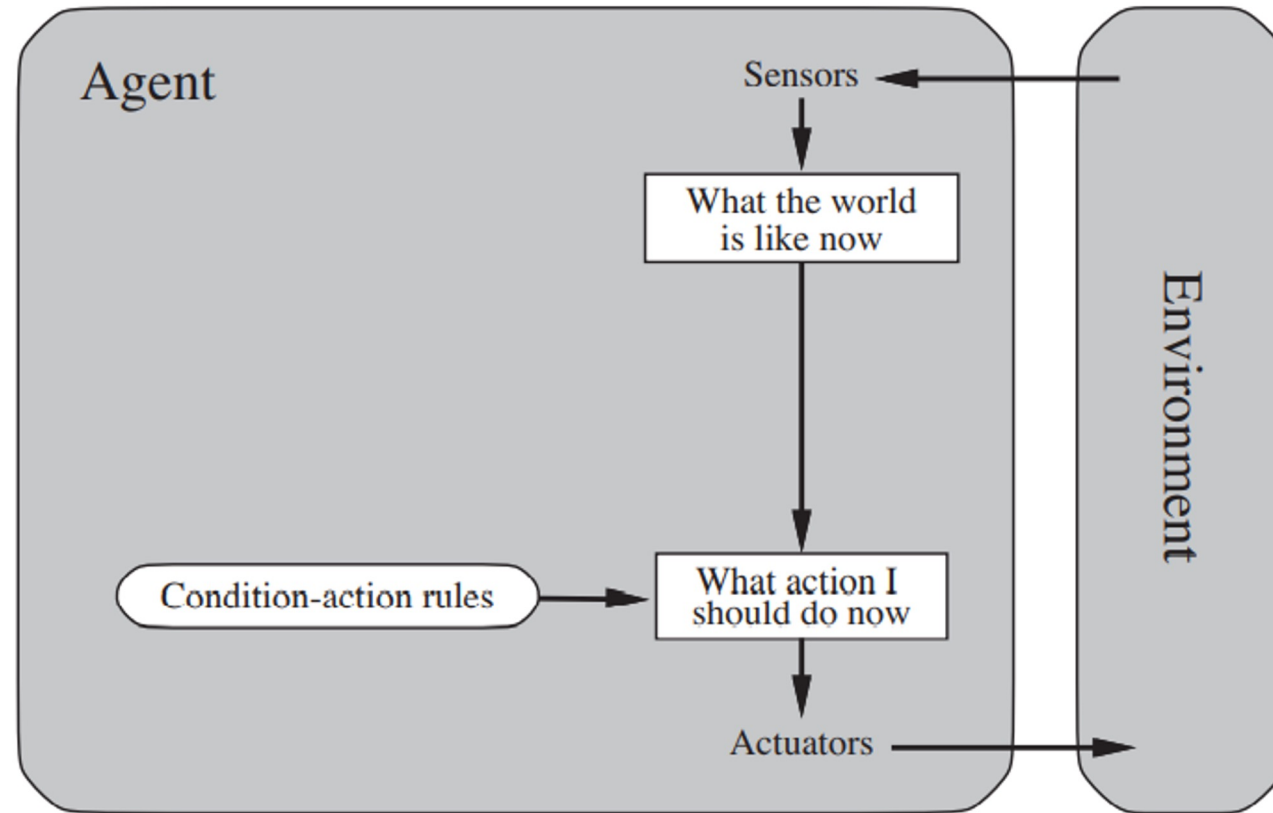
- ▶ Agents are based on simple table which entries determine the action (Percept to Percept Sequence)
- ▶ **Drawbacks:**
 - ▶ Size of the Table
 - ▶ Time to build the table
 - ▶ No Autonomy
 - ▶ Time required for learning

Simple Reflex Agent

- Simplest agent which acts according to the current percept only, pays no attention to the rest of the percept history.
- The agent function of this type relies on the **condition-action rule** – "If condition, then action."
- It makes correct decisions only if the environment is fully observable.
- **Example: iDraw**, a drawing robot which converts the typed characters into writing without storing the past data.



Simple Reflex Agent

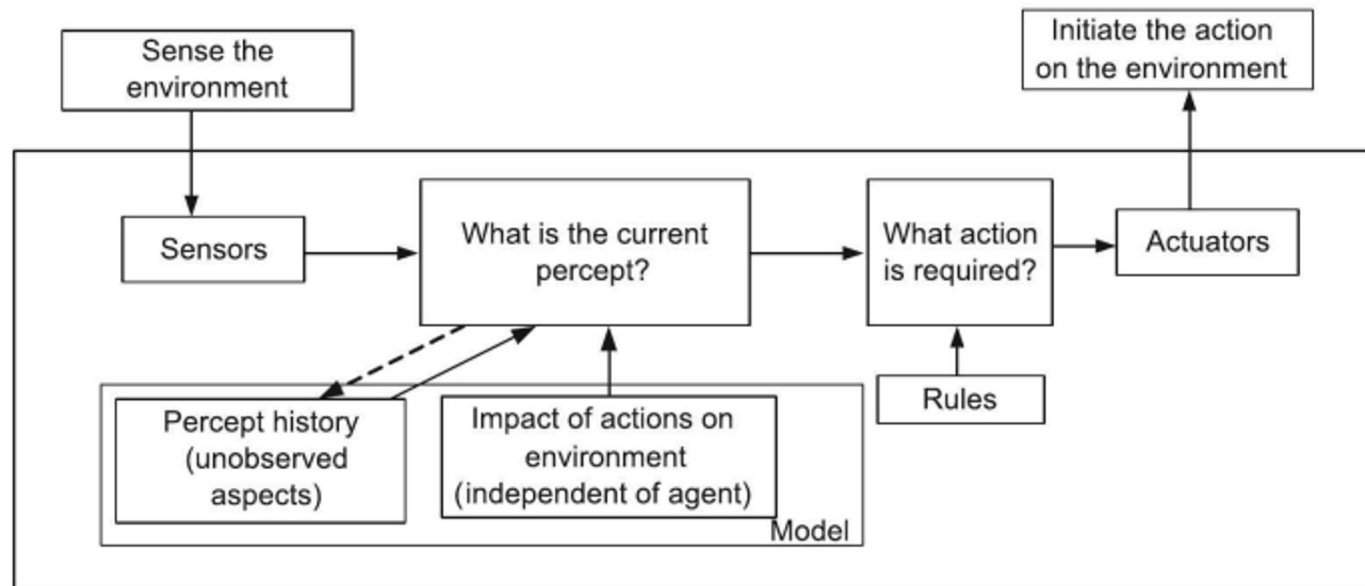


function REFLEX-VACUUM-AGENT(*[location, status]*) **returns** an action

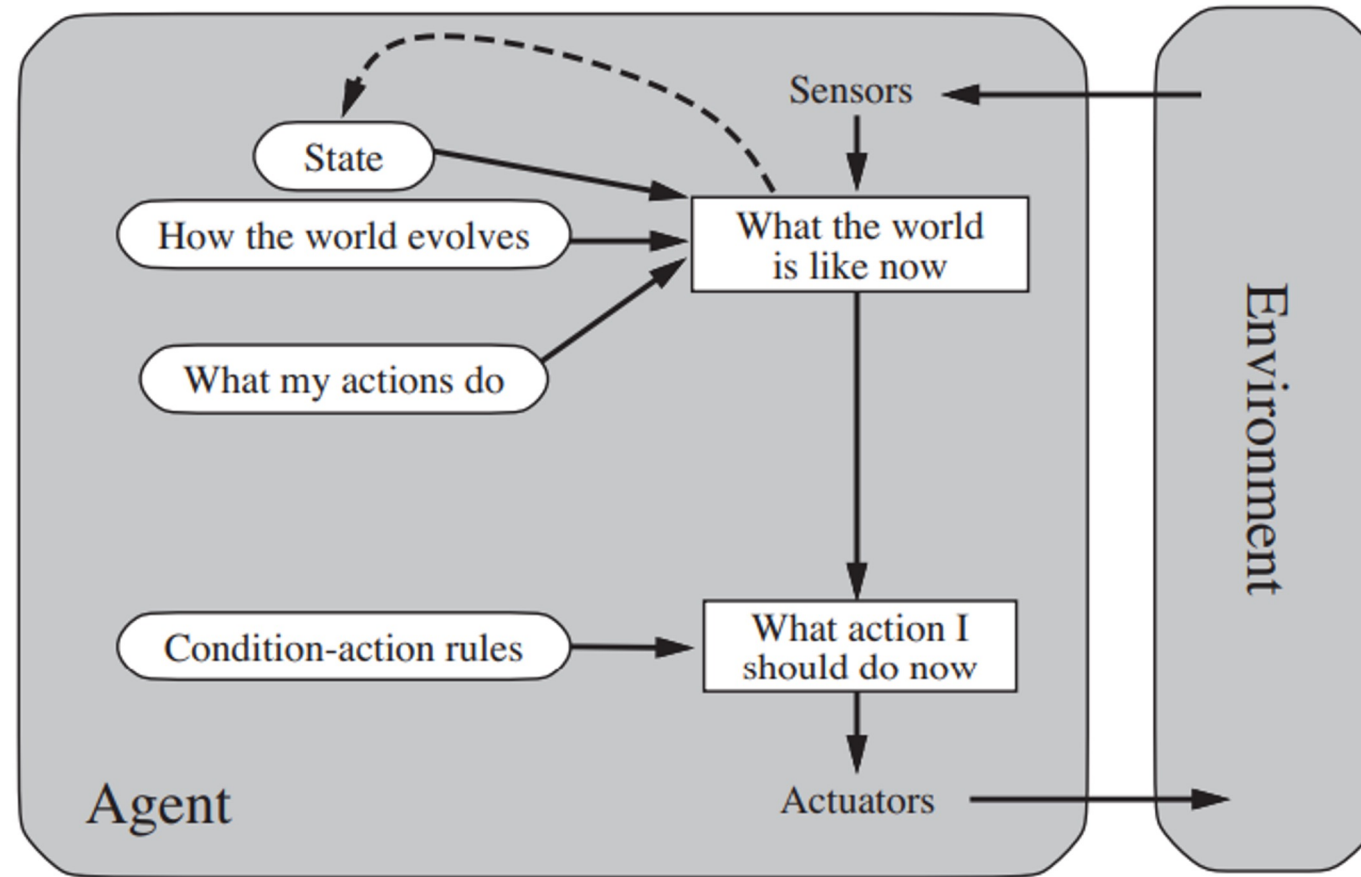
if *status* = *Dirty* **then return** *Suck*
else if *location* = *A* **then return** *Right*
else if *location* = *B* **then return** *Left*

Model-based reflex agents

- These type of agents can handle partially observable environments by maintaining some internal states. (**Agents with memory**)
- The internal state depends on the percept history, which reflects at least some of the unobserved aspects of the current state.
- Therefore, as time passes, the internal state needs to be updated which requires two types of knowledge or information to be encoded in an agent program i.e., the evolution of the world on its own and the effects of the agent's actions.



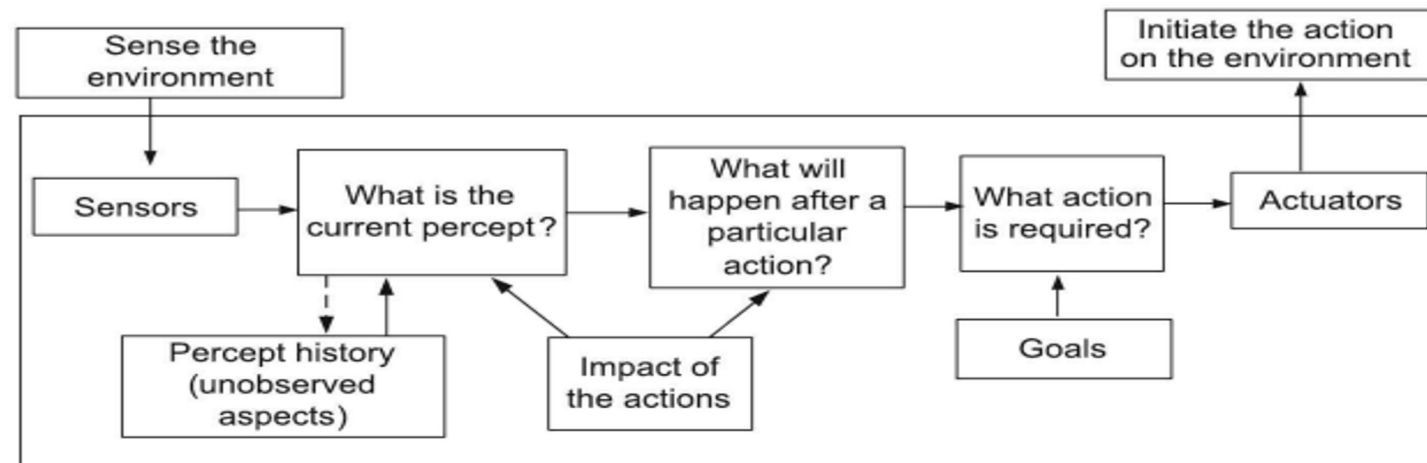
Model Based Reflex Agent



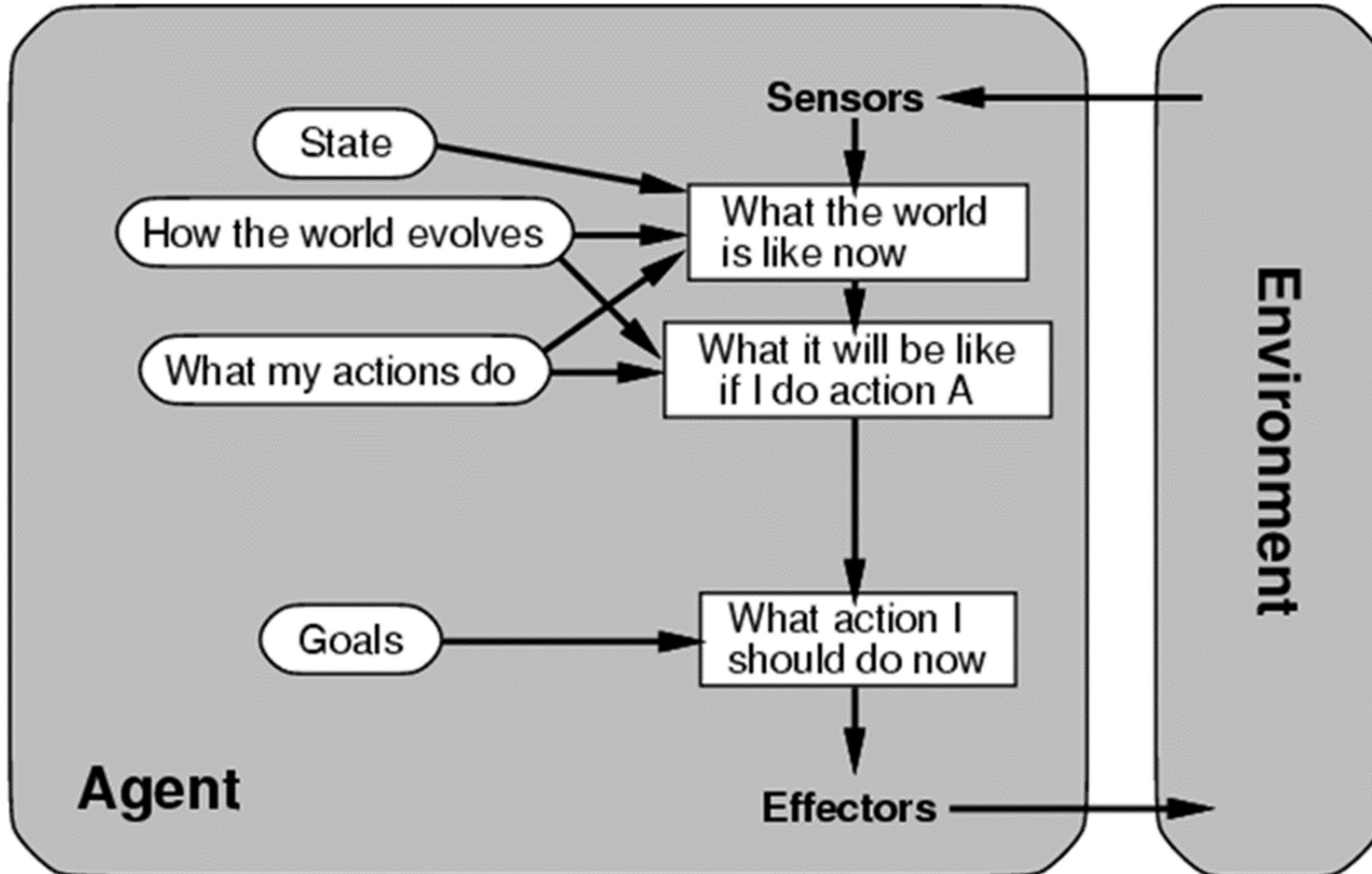
Example: When a person walks in a lane, he maps the pathway in his mind.

Goal-based agents

- Choose actions so as to achieve a (given or computed) goal.
- A goal is a description of a desirable situation.
- Keeping track of the current state is often not enough - need to add goals to decide which situations are good
- **Deliberative** instead of **reactive**.
- May have to consider long sequences of possible actions before deciding if goal is achieved – involves consideration of the future, *“what will happen if I do...?”*

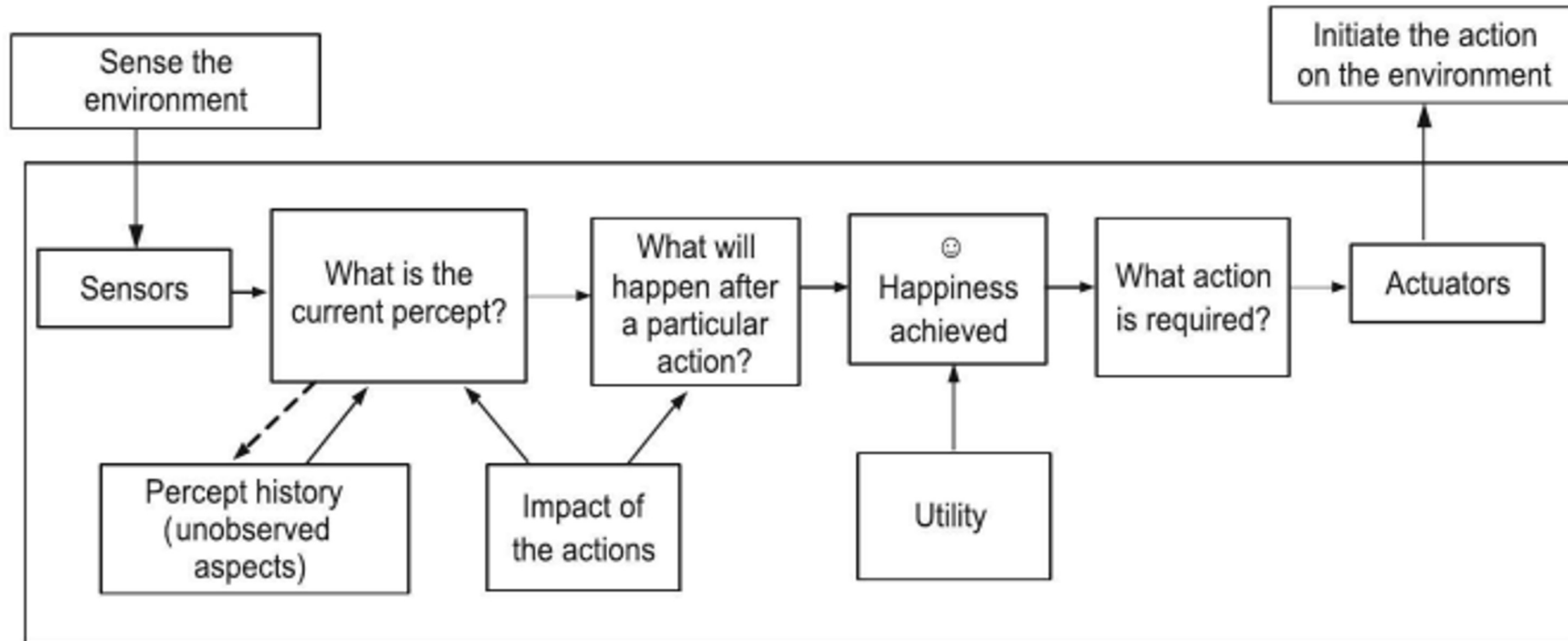


(3) Architecture for goal-based agent

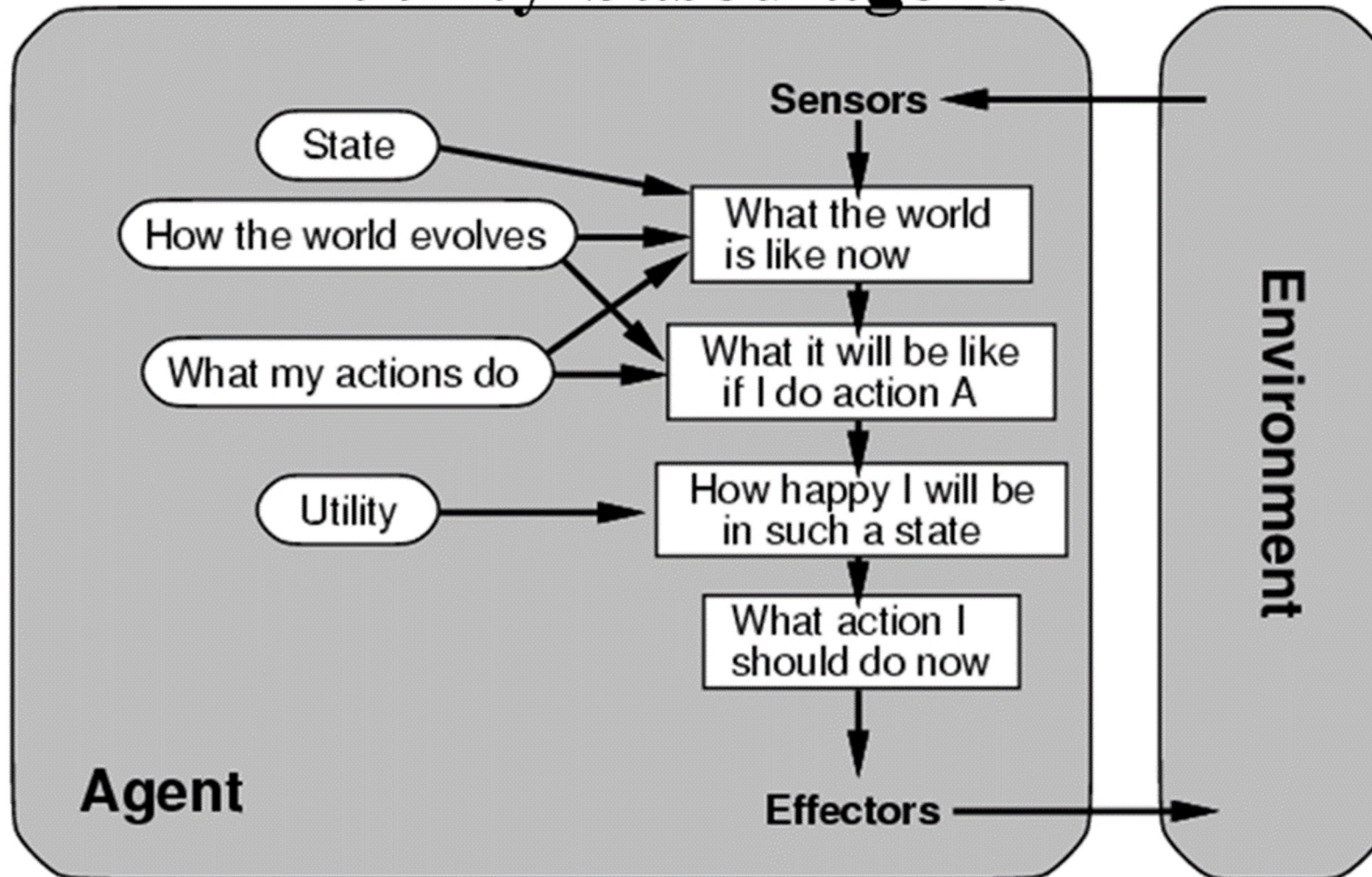


Utility Based Agent

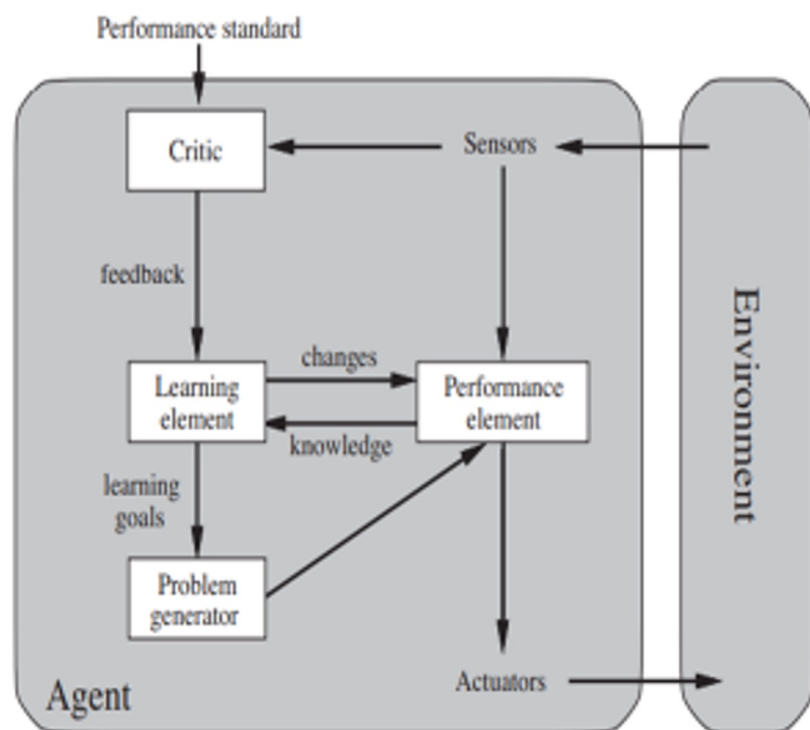
- When there are multiple possible alternatives, how to decide which one is best?
- A goal specifies a crude distinction between a happy and unhappy state, but often need a more general performance measure that describes “degree of happiness.”
- Utility function U : **State** $\rightarrow$ Reals indicating a measure of success or happiness when at a given state.
- Allows decisions comparing choice between conflicting goals, and choice between likelihood of success and importance of goal (if achievement is uncertain).



(4) Architecture for a complete utility-based agent



Learning agents



- A learning agent can be divided into four conceptual components
 - **Learning element** -Responsible for **making improvements**
 - **Performance element** -Responsible for **selecting external actions** (previously considered to be the entire agent)
 - **Critic** - The learning element uses feedback from the critic on **how the agent is doing and determines how the performance element should be modified** to do better in the future.
 - **Problem generator** - Responsible for **suggesting actions that will lead to new and informative experiences.**

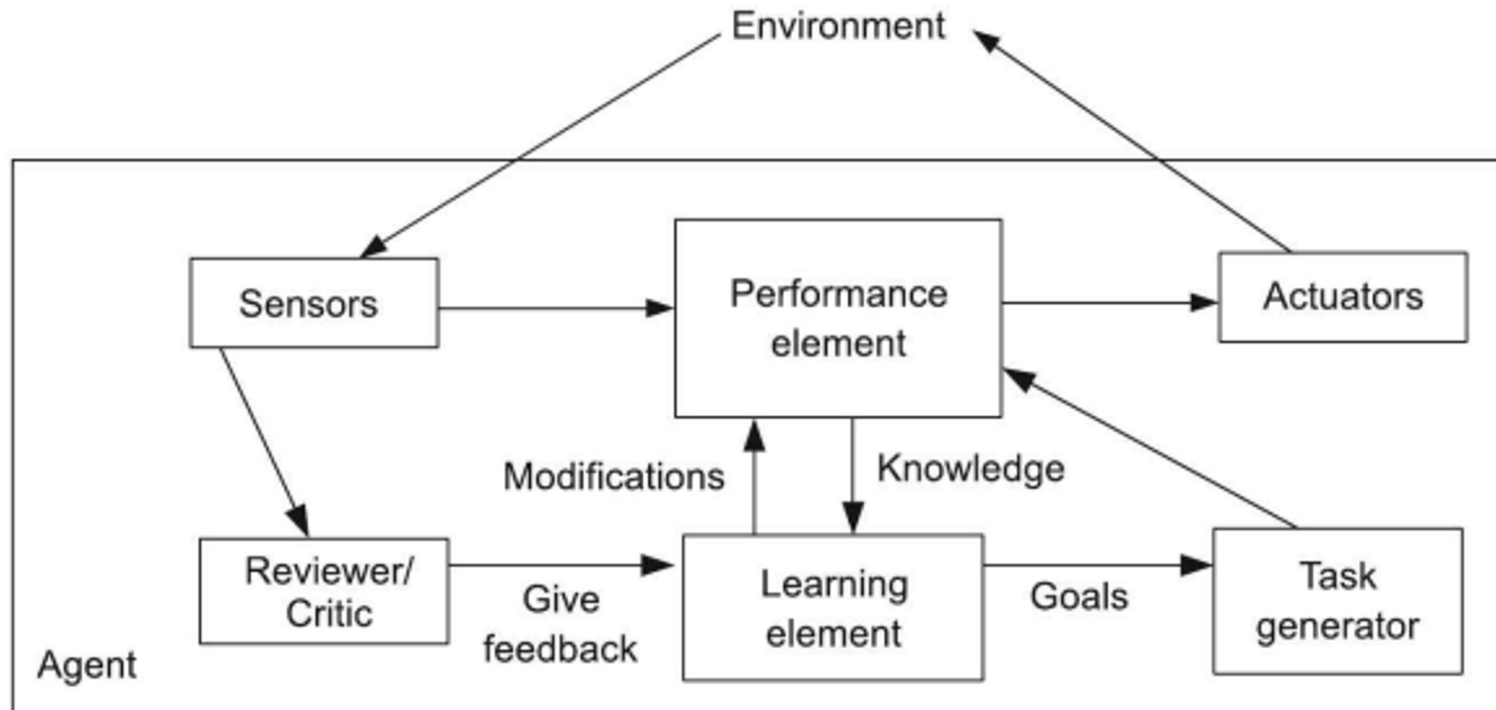


Figure 5.7 Learning agent.

THANK YOU